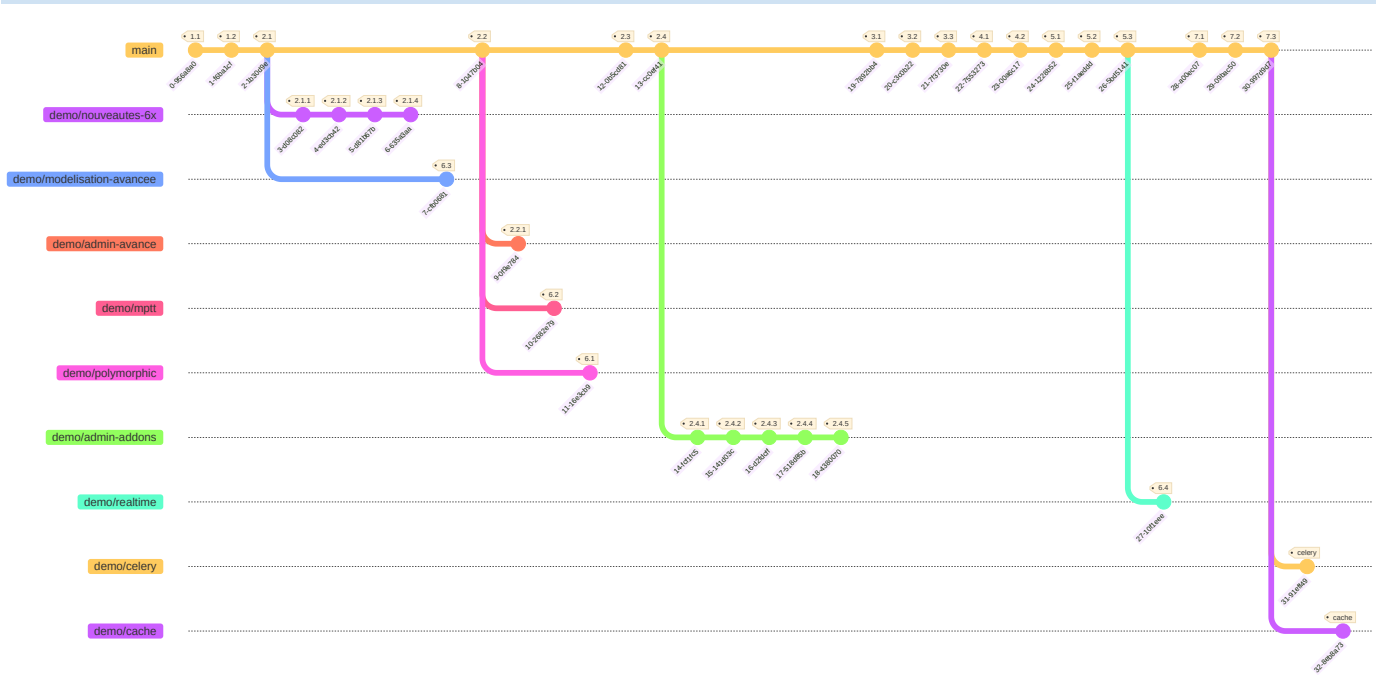


Architecture du fil rouge — taskflow

Formation Django avancé (Dawan) · application support construite pas à pas · Django 6.1 · annexe technique — carte des branches et des étapes

Le dépôt est organisé en **étapes numérotées** (`etape-X.Y`) sur la branche `main` , et en **branches de démonstration** `demo/*` qui partent chacune d'un tag et n'ajoutent que leur fonctionnalité (sans reprendre la suite de `main`). Repère rapide des phases : 1-2 = J1, 3 = J1/J2, 4 = J2, 5 = J3, 6-7 = J4.

Diagramme du dépôt



Étapes de la branche `main`

Phase	Tag	Contenu
1 — Mise en place	etape-1.1	Squelette Django 6.1, configuration, dépendances, page d'accueil publique et vues d'authentification.
	etape-1.2	Docker et docker-compose : PostgreSQL, MySQL (au choix via <code>DB_ENGINE</code>), Mailpit, Redis, Adminer.
2 — Modèles et administration	etape-2.1	Modèles du domaine (Project, Task, Status, Assignment...), enums, mixins, migration initiale ; <code>de lay</code> en <code>GeneratedField</code> .
	etape-2.2	Interface d'administration de base (registrations, <code>list_display</code> , filtres, inlines, actions) — sans personnalisation métier .
	etape-2.3	Données de démonstration : commande <code>seed_demo</code> (comptes alice/bob/charlie staff + équipe, root superadmin).
	etape-2.4	Managers et querysets métier (trois approches) + clés naturelles (<code>natural_key</code>).
3 — Validation, signaux, PDF	etape-3.1	Signaux : slug automatique, horodatage write-once de la fin réelle, métriques.
	etape-3.2	Export PDF de la fiche tâche (WeasyPrint).
	etape-3.3	Génération PDF en tâche de fond avec <code>django.tasks</code> (Django 6.0).
4 — API et front htmx	etape-4.1	API Django-Ninja : tableau Kanban et déplacement de tâches.

	etape-4.2	Front htmx : recherche live, changement de statut inline, champs dépendants, formset.
5 — Front moderne	etape-5.1	Cascade géographique htmx (django-cities-light).
	etape-5.2	Autocomplétion de ville côté serveur (Ninja).
	etape-5.3	Îlots Vue + Vite (Kanban, cascade) + Tom Select.
7 — Tests et finitions	etape-7.1	Tests : pytest-django + pytest-bdd + pyhamcrest.
	etape-7.2	Notes techniques et récapitulatif des branches (README).
	etape-7.3	Ne pas versionner <code>node_modules</code> (garder <code>package-lock.json</code>).

La phase 6 (demos avancées) et les sous-étapes `2.x.y` vivent sur des branches `demo/*`.

Branches de démonstration

demo/nouveautes-6x — part de etape-2.1
Nouveautés récentes de Django appliquées au modèle <code>Task / Assignment</code>, une par sous-étape : etape-2.1.1 db_default (Django 5.0) sur <code>Task.is_billable</code>. etape-2.1.2 CheckConstraint (Django 5.1) sur <code>Task.progress</code> (0–100). etape-2.1.3 Index composite + partiel sur <code>Task</code>. etape-2.1.4 CompositePrimaryKey (Django 5.2) sur <code>Assignment</code> — et ses limites (pas cible de FK, admin partiel), d'où le choix de <code>main</code> de garder une table d'association classique.
demo/modelisation-avancee — part de etape-2.1
etape-6.3 Enums riches et champs supplémentaires (<code>ArrayField</code>, <code>DateRange</code>). <i>Runtime : PostgreSQL.</i>
demo/admin-avance — part de etape-2.2
etape-2.2.1 Administration avancée : visibilité métier via <code>get_queryset</code> (un superadmin voit tout ; un membre du staff ne voit que ses projets — owner ou membre d'équipe — et ses tâches) et <code>get_fieldsets</code> sur-mesure (jeu complet pour le superadmin, vue allégée pour le staff, avec plusieurs champs par ligne). Test autonome fourni.
demo/mptt — part de etape-2.2
etape-6.2 Arbre tâches / sous-tâches : <code>Task</code> devient <code>MPTTModel</code> (champ <code>parent</code>), admin arborescent <code>MPTTModelAdmin</code>. <i>Runtime : django-mptt.</i>
demo/polymorphic — part de etape-2.2
etape-6.1 Hiérarchie polymorphe : <code>Task</code> parent, classes filles <code>Anomaly / Action / Improvement</code> ; admin parent + enfants avec filtre polymorphe. <i>Runtime : django-polymorphic.</i>

demo/admin-addons — part de **etape-2.4**

Extensions tierces de l'admin, une par sous-étape (un paquet chacune) :

- **etape-2.4.1** Filtre **déroulant** par projet (django-admin-list-filter-dropdown).
- **etape-2.4.2** Tri des statuts par **glisser-déposer** (django-admin-sortable2).
- **etape-2.4.3** **Import / export** CSV des tâches (django-import-export).
- **etape-2.4.4** Filtre par **plage de dates** d'échéance (django-admin-rangefilter).
- **etape-2.4.5** Filtre **multi-sélection** de priorité (django-more-admin-filters).

Runtime : les 5 paquets admin ci-dessus.

demo/realtime — part de **etape-5.3**

- **etape-6.4** Kanban temps réel (Django Channels) : consumer WebSocket `BoardConsumer` , diffusion des changements au groupe d'un projet. Runtime : serveur ASGI `daphne` .

demo/celery — part de **main**

- **etape-celery** Notification e-mail **asynchrone** du *owner* à la création d'un projet : signal `post_save` sur `Project` → tâche Celery `send_project_created_email` (`.delay()`). Alternative « classique » aux tâches natives `django.tasks` de `main` .
Runtime : `celery` + Redis + un worker (`celery -A config worker`).

demo/cache — part de **main**

- **etape-cache** Trois stratégies de cache Redis sur deux bases distinctes : cache **bas niveau** d'un agrégat coûteux (`cache.get_or_set` , DB 1), **fragment** de template (`{% cache %}` , DB 1), **page entière** (`@cache_page` , DB 2). Runtime : `django-redis` + Redis.